

The Blind Flight: Synapse Drive

Autonomous Pathfinding in Unlabeled Terrains
using Deep Learning Ensembles

Ayush Raj

GitHub Repository: <https://github.com/ayu853/The-Blind-Flight---Synapse-Drive>

December 28, 2025

Contents

1	Executive Summary	3
2	Problem Statement	3
3	Methodology	3
3.1	Data Preparation	3
3.2	Model Architecture	3
3.3	Training Strategy	4
4	Advanced Inference Techniques	4
4.1	Ensemble Prediction	4
4.2	Test-Time Augmentation (Horizontal Flip)	4
4.3	The “GPS Fix” (Heuristic Post-Processing)	4
5	Path Planning & Navigation	5
5.1	Biome Detection	5
5.2	Dijkstra’s Algorithm	5
6	Results and Conclusion	5

1 Executive Summary

This project addresses the "Blind Flight" challenge, which requires an autonomous agent to navigate a grid-based map from a start point to a destination. The core difficulty lies in the fact that the terrain types for the test set are provided only as visual images without explicit labels.

To solve this, a hybrid system was developed combining Computer Vision and Graph Theory. The solution utilizes an ensemble of deep convolutional neural networks (**ResNet-18** and **EfficientNet-B0**) to classify terrain tiles, followed by a robust pathfinding algorithm (Dijkstra) that adapts to environmental "biomes" and velocity boosts. The system features advanced inference techniques, including Test-Time Augmentation (TTA) and heuristic post-processing ("GPS Fix"), ensuring high reliability and optimal path generation.

2 Problem Statement

The objective is to parse a raw map image, identify the start (Index 3) and end (Index 4) points, and classify intermediate tiles into obstacles or traversable paths (with varying costs). The agent must then compute the most efficient path string (e.g., "u", "d", "l", "r") to navigate the grid.

Key Constraints

- **Input:** Raw images ($W \times H$) representing a 20×20 grid.
- **Output:** A sequence of moves from Start to End.
- **Ambiguity:** Test images lack ground truth labels; terrain must be inferred visually.

3 Methodology

3.1 Data Preparation

The training dataset consisted of full map images paired with JSON files containing the 20×20 ground truth grid.

- **Slicing Strategy:** Each map image was dynamically sliced into 400 individual tiles (corresponding to the grid cells).
- **Labeling:** Each tile was assigned a label (0-4) based on the JSON grid coordinates.
- **Result:** This process generated a robust dataset of labeled 224×224 tile images, allowing the problem to be treated as a standard Image Classification task.

3.2 Model Architecture

To ensure high accuracy and generalization, an ensemble of two distinct architectures was employed:

1. **ResNet-18:** Chosen for its residual learning framework, which prevents vanishing gradients and captures deep textural features efficiently.

2. **EfficientNet-B0:** Selected for its superior parameter efficiency and ability to capture complex spatial hierarchies with lower computational cost.

Both models were initialized with pretrained weights (ImageNet) to leverage transfer learning, significantly reducing convergence time.

3.3 Training Strategy

- **Loss Function:** CrossEntropyLoss was used for multi-class classification.
- **Optimizer:** Adam optimizer with a learning rate of 0.0005.
- **Class Imbalance Handling:** A `WeightedRandomSampler` was implemented to over-sample rare classes (like Start/End points) and undersample common background tiles, preventing model bias.
- **Scheduler:** `ReduceLROnPlateau` was used to decay the learning rate when validation accuracy plateaued.

4 Advanced Inference Techniques

4.1 Ensemble Prediction

During inference, the test image is sliced into tiles. Both ResNet and EfficientNet predict probabilities for every tile. The final prediction is the arithmetic mean of both models' outputs:

$$P_{final} = \frac{P_{ResNet} + P_{EfficientNet}}{2} \quad (1)$$

This ensemble approach smooths out individual model errors and reduces variance.

4.2 Test-Time Augmentation (Horizontal Flip)

To further improve robustness, Test-Time Augmentation (TTA) was applied.

1. **Forward Pass:** The model predicts on the original image.
2. **Augmented Pass:** The image is horizontally flipped, fed into the model, and the output is flipped back to the original orientation.
3. **Aggregation:** The two probability maps are averaged.

This technique ensures that the model's prediction is invariant to orientation, correcting errors where the model might miss a feature due to its specific alignment.

4.3 The "GPS Fix" (Heuristic Post-Processing)

A critical failure mode in pathfinding is the absence of a Start or End node due to classification errors. A "GPS Fix" algorithm was implemented to guarantee valid graph topology:

- **Logic:** The system scans the predicted grid. If the 'Start' (Class 3) or 'End' (Class 4) labels are missing, the algorithm identifies the tile with the highest *probability* for that class (even if it wasn't the top choice) and forcibly assigns the label.
- **Impact:** This ensures the Dijkstra algorithm always receives a solvable grid, preventing runtime crashes.

5 Path Planning & Navigation

5.1 Biome Detection

Traversal costs are not static; they depend on the environment. A heuristic pixel analysis analyzed the global color distribution of the map to classify it into one of three biomes:

- **Forest:** High concentration of green pixels. High movement cost.
- **Desert:** High yellow/orange intensity. Medium movement cost.
- **Lab:** Default environment. Low movement cost.

5.2 Dijkstra’s Algorithm

The navigation is modeled as a shortest-path problem on a weighted graph.

- **Nodes:** The 20×20 grid cells.
- **Edges:** Connections to adjacent cells (Up, Down, Left, Right).
- **Weights:** Calculated dynamically:

$$Weight = BaseCost_{Biome} - VelocityBoost$$

Dijkstra’s algorithm traverses this graph to find the sequence of moves with the lowest total cost.

6 Results and Conclusion

The proposed solution successfully automates the navigation process. The combination of ensemble deep learning provided high-accuracy terrain recognition, while the GPS Fix and TTA ensured 100% submission validity. The adaptive pathfinding logic allowed the agent to exploit velocity boosts and avoid high-cost terrain effectively.

Key Metrics

- **Training Accuracy:** $> 99\%$ (Converged within 8 epochs)
- **Inference Speed:** $\approx 2.02s$ per map (including TTA and pathfinding)

This project demonstrates the efficacy of combining varied neural architectures with heuristic logic to solve complex, multi-stage autonomous control problems.